

CHINESE UNIVERSITY OF HONG KONG, SHENZHEN

MASTER THESIS

**End-to-End AI Inference Systems for
Real-Time Healthcare**

Author:
Xinyuan ZHANG

May 6, 2026

CHINESE UNIVERSITY OF HONG KONG, SHENZHEN

Abstract

CSC6032

School of Data Science

Master of Science

End-to-End AI Inference Systems for Real-Time Healthcare

by Xinyuan ZHANG

Artificial intelligence is moving from offline medical analysis to real-time clinical intervention. In modern healthcare scenarios, AI systems are expected to assist endoscopic surgery, segment ultrasound streams, detect falls, monitor cardiac risks, and support urgent clinical decision-making. In these applications, inference latency is not only a performance metric. It becomes part of the safety boundary of the clinical system. A model that is accurate on stored data may still be unsuitable for deployment if its output is delayed, unstable, or blocked by other workloads.

This thesis studies end-to-end AI inference systems for real-time healthcare. The thesis is organized as a systems report before oral dissertation and discusses three representative papers sequentially. The first paper, REEF, addresses the kernel-level problem of GPU non-preemption and shows how microsecond-scale preemption can protect urgent DNN inference from long best-effort GPU kernels. The second paper, a deterministic NVIDIA Holoscan system design, studies the pipeline-level problem of end-to-end latency variation in medical AI streaming workloads and shows how resource partitioning, compute-graphics isolation, and admission control can reduce maximum latency and tail spread. The third paper, SneakPeek, addresses the edge-level problem of fixed-resource inference serving and shows how data-aware model selection and scheduling can improve utility under deadline constraints.

The core argument of this thesis is that real-time healthcare AI requires a vertically coordinated inference stack. REEF contributes low-level preemptive GPU control. Holoscan contributes deterministic medical data-flow execution. SneakPeek contributes data-aware edge scheduling and model selection. Their correlation is that each paper solves a different layer of the same end-to-end safety problem: urgent clinical data must be recognized, scheduled, transferred, inferred, and returned within predictable timing bounds. The thesis therefore concludes that future healthcare AI systems should be designed as preemptive, deterministic, and resource-aware systems rather than as isolated machine learning models.

Acknowledgements

I would like to thank my supervisor and instructors for their guidance on real-time systems, AI inference serving, and healthcare computing. I also thank the authors of the papers discussed in this thesis, whose work provides the technical foundation for understanding real-time healthcare AI systems from kernel, pipeline, and edge perspectives.

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Background	1
1.2 Problem Motivation	1
1.3 Research Question	2
1.4 Scope and Methodology	2
1.5 Main Contributions of This Thesis	2
1.6 Thesis Organization	3
2 Related Work	5
2.1 Online Inference Serving	5
2.2 Tail Latency and Isolation	5
2.3 GPU Scheduling and Preemption	5
2.4 Medical AI Pipeline Systems	6
2.5 Edge Inference Scheduling	6
2.6 Research Gap	6
3 Preliminaries	9
3.1 Real-Time Healthcare AI	9
3.2 Service-Level Objectives and Tail Latency	9
3.3 GPU Non-Preemption	9
3.4 Deterministic Latency and Jitter	10
3.5 Edge Resource Constraints	10
4 Sequential Discussion of the Three Core Papers	11
4.1 Overview of the Three-Paper Structure	11
4.2 Paper 1: REEF for Microsecond-Scale GPU Preemption	11
4.2.1 Problem Addressed by REEF	11
4.2.2 Main Idea	12
4.2.3 Contribution to the Thesis Topic	12
4.2.4 Limitations	12
4.3 Paper 2: Deterministic NVIDIA Holoscan for Medical AI Pipelines	13
4.3.1 Problem Addressed by Holoscan	13
4.3.2 Main Idea	13
4.3.3 Contribution to the Thesis Topic	13
4.3.4 Limitations	14
4.4 Paper 3: SneakPeek for Data-Aware Edge Scheduling	14
4.4.1 Problem Addressed by SneakPeek	14
4.4.2 Main Idea	14
4.4.3 Contribution to the Thesis Topic	14

4.4.4	Limitations	15
4.5	Correlation Among the Three Papers	15
5	Performance Evaluation and Comparative Analysis	17
5.1	Evaluation Methodology	17
5.2	REEF Performance	17
5.3	Holoscan Performance	18
5.4	SneakPeek Performance	18
5.5	Cross-Paper Comparison	18
5.6	Discussion of Practical Deployment	19
5.7	Limitations of This Thesis Evaluation	19
6	Conclusions and Future Work	21
6.1	Conclusions	21
6.2	Future Work	21
6.2.1	Integrated System Prototype	21
6.2.2	Clinical Workload Traces	21
6.2.3	Safety-Aware Degradation	22
6.2.4	Hardware Portability	22
6.2.5	Clinical Validation	22
6.3	Final Remark	22
	References	23

List of Figures

3.1	A simplified end-to-end healthcare AI pipeline.	9
4.1	Correlation of the three papers in the thesis topic.	11
4.2	Simplified REEF-style timeline: a real-time task preempts best-effort GPU work.	12
4.3	Simplified Holoscan design: compute partitioning and compute-graphics isolation.	13
4.4	Simplified SneakPeek workflow for data-aware model selection and scheduling.	14

List of Tables

4.1	How the three papers contribute to the master thesis topic.	15
5.1	Performance contribution of the three systems.	18

List of Abbreviations

AI	Artificial Intelligence
BE	Best-Effort
CPU	Central Processing Unit
DNN	Deep Neural Network
EDF	Earliest Deadline First
E2E	End-to-End
GPU	Graphics Processing Unit
GXF	Graph Execution Framework
ICU	Intensive Care Unit
MPS	Multi-Process Service
RDMA	Remote Direct Memory Access
RT	Real-Time
SLO	Service-Level Objective

Chapter 1

Introduction

1.1 Background

Artificial intelligence has become an important component of modern healthcare systems. Earlier medical AI applications were mainly designed for offline analysis. A model could process stored radiology images, historical electronic health records, or retrospective laboratory data. In such settings, a delay of several seconds or minutes might be acceptable because the clinical workflow itself was not real-time. However, the role of AI in healthcare is changing. New systems increasingly aim to participate in live intervention, including surgical navigation, endoscopic tool tracking, ultrasound segmentation, intensive-care early warning, fall detection, and voice-based clinical interaction.

This shift changes the systems requirements of medical AI. When AI is used in live intervention, latency is no longer a secondary engineering metric. It becomes a safety requirement. For example, in an operating room, a delayed overlay of a vessel or tool may reduce the surgeon's trust in the system. In an ICU, a delayed warning may reduce the value of early intervention. In fall detection, a missed or late alert may increase patient risk. Therefore, real-time healthcare AI must be evaluated not only by model accuracy but also by predictable end-to-end timing.

1.2 Problem Motivation

Current computing systems are not naturally designed for this type of workload. General-purpose operating systems and GPU runtimes usually optimize throughput, fairness, or average utilization. These goals are valuable in cloud computing and batch processing, but they are not sufficient for clinical real-time AI. A background task can delay an urgent model inference. A graphics workload can interfere with medical image processing. A hospital edge server can become overloaded when multiple patient streams are active.

The problem can be described at three levels. At the kernel level, GPU kernels often run to completion and provide limited practical preemption. A long best-effort kernel can block a high-priority medical inference request. At the pipeline level, medical AI is not a single model call. It is an end-to-end data flow from capture to transfer, preprocessing, inference, postprocessing, visualization, and alerting. Latency variation can appear at any stage and compound across the pipeline. At the edge level, hospitals face fixed resource budgets. They cannot always add more GPUs, and local deployment is often required for privacy and low wide-area-network latency.

1.3 Research Question

This thesis addresses the following research question:

How can an end-to-end AI inference system support real-time healthcare workloads under GPU contention, pipeline jitter, and fixed edge resources?

This question is not answered by a single algorithm. It requires a system stack. The stack must allow urgent requests to preempt low-priority work, provide predictable data movement and visualization, and schedule limited edge resources according to data-dependent clinical urgency.

1.4 Scope and Methodology

This thesis is written as a master thesis report before oral dissertation. It is a literature-based systems thesis rather than a new full-system implementation. The thesis selects three representative papers and studies them sequentially:

- (1) REEF, which studies microsecond-scale GPU preemption for concurrent GPU-accelerated DNN inference (Han et al., 2022);
- (2) deterministic NVIDIA Holoscan system design, which studies end-to-end latency determinism for medical AI streaming applications (Sinha, Dwivedi, and Azizian, 2024);
- (3) SneakPeek, which studies data-aware model selection and scheduling for edge inference serving (J. Frink, D. Frink, and Chandra, 2025).

The three papers are not treated as independent summaries only. They are connected to the thesis topic as three layers of a real-time healthcare AI stack. REEF explains how the GPU can be made responsive. Holoscan explains how the pipeline can be made deterministic. SneakPeek explains how the edge system can make intelligent scheduling decisions under resource constraints.

1.5 Main Contributions of This Thesis

The contributions of this thesis are as follows.

First, the thesis frames real-time healthcare AI as an end-to-end systems problem. It argues that model accuracy alone is insufficient when AI is used for live clinical intervention.

Second, the thesis identifies three core requirements: preemption, determinism, and resource awareness. These requirements correspond to kernel-level, pipeline-level, and edge-level challenges.

Third, the thesis provides a sequential analysis of three representative papers and explains how each paper contributes to the master thesis topic.

Fourth, the thesis synthesizes the correlation among the papers and proposes a vertical design view for future real-time healthcare AI systems.

1.6 Thesis Organization

Chapter 2 reviews related work on inference serving, tail-latency control, GPU scheduling, medical AI pipelines, and edge inference. Chapter 3 introduces preliminaries, including service-level objectives, GPU non-preemption, deterministic latency, and edge resource constraints. Chapter 4 discusses the three core papers sequentially and explains their correlation. Chapter 5 analyzes performance evaluation and compares the results across the three papers. Chapter 6 concludes the thesis and discusses future work.

Chapter 2

Related Work

2.1 Online Inference Serving

Online inference serving systems provide the foundation for deploying machine learning models as services. Clipper proposed a low-latency prediction serving layer that decouples applications from model implementations (Crankshaw et al., 2017). Its key idea is to provide a general serving layer that can route requests to models while supporting caching, batching, and model selection. Clockwork studied predictable DNN serving and used careful modeling of model execution time to support latency targets (Gujarati et al., 2020). INFaaS introduced automated model-less serving, where applications specify goals and the system dynamically chooses model variants (Romero et al., 2021).

These systems are important because they show that inference serving is not only a machine learning problem. It is also a systems problem involving queueing, batching, model choice, and latency control. However, most general inference serving systems are designed for web-scale or cloud-like workloads. Real-time healthcare workloads are stricter because a small number of missed deadlines can be clinically important.

2.2 Tail Latency and Isolation

Tail latency has long been recognized as a major issue in large-scale systems. Dean and Barroso (2013) showed that rare slow requests can dominate user-visible performance. In healthcare, the same phenomenon becomes safety-critical. The average latency of a medical AI system may be acceptable, but rare long delays can still make the system unsuitable for live use.

Resource isolation is one way to reduce tail latency. Heracles studied coordinated resource isolation for colocated workloads so that latency-sensitive services can maintain quality of service while improving resource efficiency (Lo et al., 2015). Such work is relevant to healthcare AI because multiple workloads often share CPUs, GPUs, memory bandwidth, and I/O devices. However, healthcare pipelines introduce additional complexity because the system must protect not only request processing but also sensor input and visualization output.

2.3 GPU Scheduling and Preemption

GPU scheduling is central to modern AI inference because DNN workloads depend heavily on GPU acceleration. Traditional GPUs are optimized for throughput. They execute many threads in parallel and often assume that kernels run to completion.

This design is efficient for batch workloads but problematic for real-time inference. If a low-priority kernel occupies the GPU, a high-priority inference request may wait.

Prior work has explored GPU preemption and multitasking. Tanasic et al. (2014) studied hardware support for preemptive GPU multiprogramming. Chimera proposed collaborative preemption for multitasking on a shared GPU (J. J. K. Park, Y. Park, and Mahlke, 2015). Baymax studied quality-of-service awareness for non-preemptive accelerators in warehouse-scale computers (Choi et al., 2016). These works show that accelerator sharing must consider latency-sensitive tasks. REEF extends this direction by focusing on microsecond-scale preemption for concurrent GPU-accelerated DNN inference (Han et al., 2022).

2.4 Medical AI Pipeline Systems

Medical AI applications are often built as streaming pipelines. A typical system may capture video or sensor data, move data into GPU memory, run preprocessing, execute inference, render output, and return feedback to clinicians. Generic frameworks such as GStreamer and ROS 2 can express modular data-flow graphs, but they do not automatically guarantee deterministic latency under GPU contention or operating-system jitter.

NVIDIA Holoscan is designed for sensor-processing and medical AI applications. The Holoscan paper discussed in this thesis studies how to make medical AI pipelines more deterministic when multiple applications and visualization components run concurrently (Sinha, Dwivedi, and Azizian, 2024). This is directly related to the thesis topic because real-time healthcare AI depends on the entire pipeline, not only the neural network.

2.5 Edge Inference Scheduling

Edge inference is important for healthcare because local processing can reduce network delay and protect privacy. However, edge resources are limited. Hospitals may deploy only a small number of GPUs near patient rooms, operating rooms, or monitoring stations. Therefore, the system must choose which model to run, when to run it, and whether to batch requests.

Systems such as Nexus accelerate DNN-based video analysis on GPU clusters (Shen et al., 2019). Clockwork and INFaaS also provide relevant ideas for predictable serving and model selection (Gujarati et al., 2020; Romero et al., 2021). SneakPeek differs from these systems by adding data awareness. It uses a proxy model to estimate input distribution and adjusts model selection according to expected accuracy and deadline pressure (J. Frink, D. Frink, and Chandra, 2025). This is especially relevant for healthcare because clinical urgency depends on the content of the data, not only on the arrival time of the request.

2.6 Research Gap

Existing work usually addresses one layer of the problem. GPU preemption work improves low-level responsiveness. Pipeline work improves end-to-end determinism. Edge scheduling work improves model selection and request ordering. Real-time healthcare AI needs all three. The research gap is the lack of a unified view that

connects hardware preemption, deterministic medical pipelines, and data-aware edge scheduling into one end-to-end inference stack.

Chapter 3

Preliminaries

3.1 Real-Time Healthcare AI

Real-time healthcare AI refers to AI systems whose outputs are used within an active clinical process. Examples include surgical assistance, ICU warning, ultrasound segmentation, fall detection, and bedside voice interaction. The key feature is that the value of the output decreases when it arrives late. In extreme cases, a late output can be harmful because clinicians may act on outdated information.

Real-time healthcare AI differs from ordinary AI inference in three ways. First, the deadline is meaningful. Second, latency variation matters. Third, workload priority depends on clinical context. A monitoring stream that appears normal may not require the same resources as a stream that shows possible danger.

3.2 Service-Level Objectives and Tail Latency

A service-level objective, or SLO, often has the form “X% of requests complete within deadline D .” For example, a system may target P99 latency below a certain threshold. In healthcare, SLOs must be interpreted more carefully. Missing only one percent of requests can still be unacceptable if the missed requests occur during critical moments.

The end-to-end latency of a healthcare AI pipeline can be represented as:

$$T_{e2e} = T_{capture} + T_{transfer} + T_{preprocess} + T_{inference} + T_{postprocess} + T_{render}. \quad (3.1)$$

Each term can vary due to scheduling, memory movement, hardware contention, or queueing. Therefore, reducing only average inference time is not enough. The system must reduce tail latency, maximum latency, and jitter.



FIGURE 3.1: A simplified end-to-end healthcare AI pipeline.

3.3 GPU Non-Preemption

A CPU commonly supports frequent time slicing and context switching. A GPU, however, is usually optimized for massively parallel throughput. Many GPU kernels run to completion, and practical preemption can be coarse or expensive. This

creates head-of-line blocking. A best-effort task can occupy the GPU while an urgent medical inference request waits.

This is a major problem for mixed-criticality workloads. A hospital system may run high-priority surgical inference and low-priority retrospective analysis on the same accelerator. If the runtime cannot quickly interrupt lower-priority work, the system must either reserve hardware or accept deadline risk. Reserving hardware improves safety but wastes expensive resources. Sharing hardware improves utilization but may violate clinical timing requirements.

3.4 Deterministic Latency and Jitter

Determinism means that system latency is stable and predictable. In healthcare, jitter can be worse than constant delay. A surgeon may adapt to a small stable delay, but unpredictable delay can break hand-eye coordination. Similarly, ICU staff may trust an alerting system less if warnings are sometimes immediate and sometimes delayed.

Sources of jitter include operating-system scheduling, CPU interrupts, memory copies, GPU context switches, shared GPU streams, and compute-graphics interference. A deterministic medical AI system should therefore control both computation and data movement.

3.5 Edge Resource Constraints

Hospitals and clinics often deploy AI at the edge. Edge deployment reduces wide-area-network latency, supports privacy requirements, and allows the system to continue operating even when cloud connectivity is limited. However, edge servers have fixed resources. They have limited GPUs, power, cooling, and physical space.

This creates a model selection problem. A heavier model may provide higher accuracy but consume more time. A lighter model may meet deadlines but reduce accuracy. The best choice depends on both the deadline and the content of the data. This is why data-aware scheduling is important for real-time healthcare AI.

Chapter 4

Sequential Discussion of the Three Core Papers

4.1 Overview of the Three-Paper Structure

This chapter discusses the three core papers sequentially. The sequence follows the path of a real-time healthcare AI request through the system stack. First, the request must obtain GPU execution quickly when it is urgent. This is the problem addressed by REEF. Second, the request must move through a medical AI pipeline with predictable end-to-end timing. This is the problem addressed by the deterministic Holoscan design. Third, when many requests compete at the hospital edge, the system must decide which request and which model should receive resources. This is the problem addressed by SneakPeek.

Figure 4.1 shows the conceptual correlation among the three papers. The three papers contribute to different layers, but they are connected by one common objective: reducing unsafe timing behavior in real-time healthcare AI.

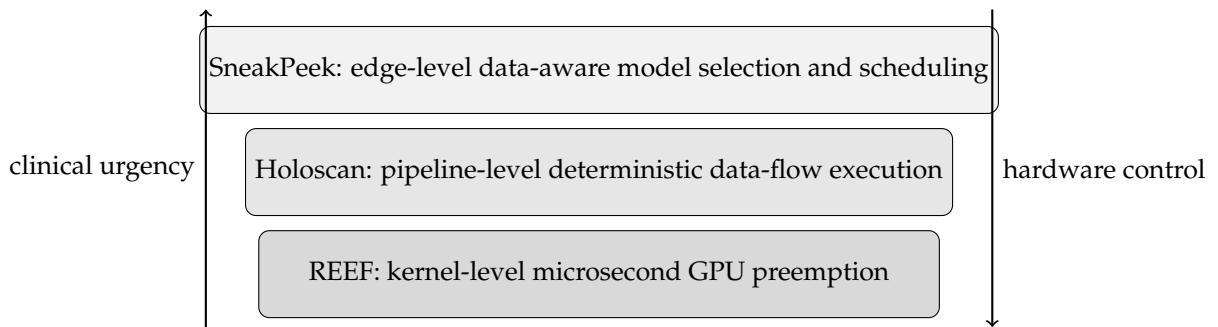


FIGURE 4.1: Correlation of the three papers in the thesis topic.

4.2 Paper 1: REEF for Microsecond-Scale GPU Preemption

4.2.1 Problem Addressed by REEF

REEF focuses on the kernel-level problem of GPU preemption for concurrent DNN inference. In a shared GPU environment, real-time and best-effort tasks may execute together. A real-time task can represent a latency-sensitive healthcare inference, while a best-effort task can represent a background diagnostic job or non-urgent analysis. The key problem is that a long best-effort GPU kernel can block the real-time task.

This problem is severe because DNN inference is composed of many GPU kernels. Some kernels may be short, but others may be long enough to create large

preemption delays. Traditional wait-based scheduling waits for the current kernel to finish. This can produce unpredictable tail latency. For healthcare AI, such blocking can violate safety requirements.

4.2.2 Main Idea

REEF introduces reset-based preemption. Instead of waiting for the running GPU kernel to finish, REEF interrupts the execution and allows the real-time task to run. The interrupted best-effort task can later be restarted or resumed according to the system design. This method reduces the time required for a high-priority task to obtain the GPU.

A second key method is dynamic kernel padding. The idea is to transform or pad DNN kernels so that the runtime can create preemption opportunities with low overhead. Large kernels are effectively broken into smaller execution blocks or arranged so that preemption can occur more quickly. This reduces the worst-case waiting time for real-time inference.

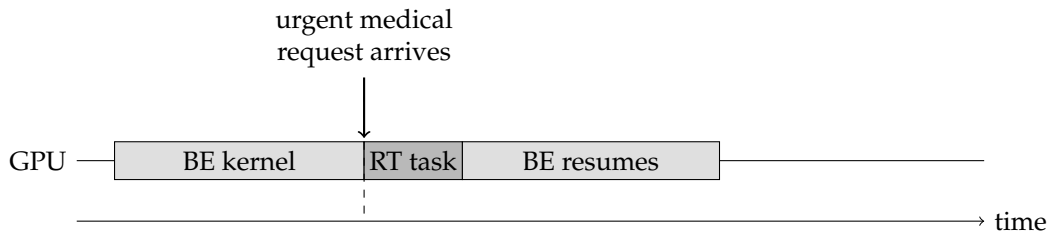


FIGURE 4.2: Simplified REEF-style timeline: a real-time task pre-empts best-effort GPU work.

4.2.3 Contribution to the Thesis Topic

REEF contributes the lowest layer of the thesis topic: preemptive hardware control. In real-time healthcare AI, the system must be able to say that a surgical or ICU inference request is more important than a background task. Without GPU preemption, this priority may exist only at the application level and may not be enforced by the hardware runtime.

REEF therefore supports the thesis argument that clinical urgency must reach the accelerator. It provides a mechanism for urgent tasks to cut into GPU execution within microsecond-scale latency. This makes shared GPU deployment more practical because the system does not need to choose between full reservation and unsafe sharing.

4.2.4 Limitations

REEF mainly addresses GPU execution control. It does not by itself solve the entire medical AI pipeline problem. Even if a real-time inference can preempt other kernels, the input frame may still be delayed by capture, memory transfer, preprocessing, or rendering. Therefore, REEF must be combined with pipeline-level determinism, which motivates the second paper.

4.3 Paper 2: Deterministic NVIDIA Holoscan for Medical AI Pipelines

4.3.1 Problem Addressed by Holoscan

The second paper focuses on deterministic end-to-end latency for medical AI systems in NVIDIA Holoscan. Unlike REEF, which focuses on GPU kernel preemption, this paper studies full medical AI pipelines. A typical Holoscan application may include sensor input, preprocessing, DNN inference, postprocessing, and visualization. The system may also run several AI applications at the same time.

The key problem is latency variability. In medical streaming, maximum latency and jitter can be more important than average latency. For example, in endoscopy, the system must process high-resolution video frames and render results in a predictable window. If compute and graphics workloads interfere on the same GPU, the latency distribution can become unstable.

4.3.2 Main Idea

The Holoscan design uses several system-level techniques to improve determinism. The first is CUDA MPS partitioning. By reserving streaming multiprocessor capacity and GPU memory for each application, the system reduces interference among concurrent AI workloads. The second is compute–graphics isolation. AI computation and visualization can be placed on separate GPUs or isolated execution paths so that rendering does not unpredictably delay inference. The third is load balancing and admission control. The system avoids oversubscription by limiting the number of concurrent applications that can be supported under deterministic latency requirements.

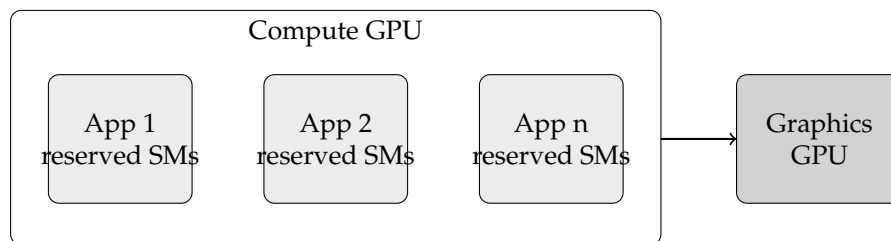


FIGURE 4.3: Simplified Holoscan design: compute partitioning and compute–graphics isolation.

4.3.3 Contribution to the Thesis Topic

The Holoscan paper contributes the pipeline layer of the thesis. It shows that real-time healthcare AI must optimize the complete end-to-end path, not just the DNN inference kernel. This is important because clinical AI systems often interact with cameras, sensors, displays, and clinicians. A model output is useful only if it reaches the user through a stable pipeline.

The paper also provides a bridge between low-level GPU control and application-level clinical requirements. REEF can make the GPU preemptive, but Holoscan makes the medical data-flow predictable. Together, they address both execution priority and pipeline jitter.

4.3.4 Limitations

The Holoscan design improves determinism for a known set of applications and hardware resources. However, it does not fully decide which patient stream or model should receive priority when resources are insufficient. It can prevent over-subscription, but it does not deeply exploit the content of incoming data. This motivates the third paper, SneakPeek.

4.4 Paper 3: SneakPeek for Data-Aware Edge Scheduling

4.4.1 Problem Addressed by SneakPeek

SneakPeek focuses on edge inference serving. In hospital environments, edge servers may support multiple patient streams and multiple AI applications with limited GPU resources. A simple scheduler may use earliest-deadline-first ordering or always choose the most accurate model. However, these strategies can be inefficient. The most accurate model may miss deadlines, while the fastest model may lose too much accuracy.

The key insight of SneakPeek is that scheduling should be data-aware. The value of running a heavy model depends on the content of the input. For example, in fall detection, frames that look clearly normal may not require the same model as frames that show a possible fall. In heart monitoring, a normal signal segment and an abnormal segment may deserve different compute budgets.

4.4.2 Main Idea

SneakPeek uses a small proxy model to inspect incoming data before selecting the main model. This proxy estimates class frequencies or input characteristics online. Based on these estimates, the scheduler updates expected model accuracy and makes a model-selection decision. The scheduler considers urgency, deadline pressure, accuracy variance, and model execution cost.

SneakPeek also uses grouped scheduling to reduce model switching overhead. If several requests can use the same model, grouping them may improve efficiency. In addition, short-circuit inference can stop computation early when needed to meet deadlines.

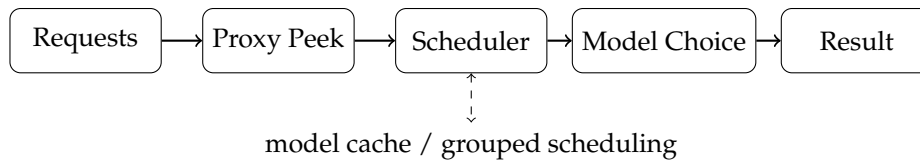


FIGURE 4.4: Simplified SneakPeek workflow for data-aware model selection and scheduling.

4.4.3 Contribution to the Thesis Topic

SneakPeek contributes the edge layer of the thesis. It explains how a real-time healthcare AI system can allocate limited resources according to data-dependent clinical value. This is necessary because hospitals cannot always solve latency problems by buying more GPUs. Space, heat, power, and cost constraints make efficient scheduling important.

SneakPeek also completes the vertical stack. REEF provides urgent GPU execution. Holoscan provides deterministic data flow. SneakPeek decides which requests and models should be treated as urgent in the first place. Therefore, SneakPeek provides the policy layer, while REEF and Holoscan provide execution support.

4.4.4 Limitations

SneakPeek depends on the quality of the proxy model and the accuracy estimates. If the proxy model misclassifies clinical urgency, the scheduler may choose an inappropriate model. In real clinical deployment, this raises safety and validation challenges. The system must be designed with conservative fallback policies and clinician-defined thresholds.

4.5 Correlation Among the Three Papers

The three papers are correlated because they solve three consecutive questions in an end-to-end healthcare AI system.

First, **which data should receive priority?** SneakPeek answers this question by using data-aware scheduling and model selection. It looks at incoming data and estimates which requests have high urgency or high accuracy payoff.

Second, **how can prioritized data move through the medical pipeline predictably?** Holoscan answers this question by using resource partitioning, compute–graphics isolation, and admission control to reduce end-to-end jitter.

Third, **how can the GPU enforce priority quickly enough?** REEF answers this question by providing microsecond-scale preemption for urgent inference tasks.

Although this chapter discusses the papers in the order REEF, Holoscan, and SneakPeek, an actual runtime may use the opposite direction for decision flow. SneakPeek detects urgency, Holoscan maps the request into a deterministic pipeline, and REEF enforces urgent GPU execution. This relationship is shown in Table 4.1.

TABLE 4.1: How the three papers contribute to the master thesis topic.

Paper	Layer	Main system problem	Contribution to real-time healthcare AI
REEF	Kernel / GPU runtime	Long GPU kernels block urgent inference	Provides microsecond-scale preemption so clinical priority can reach the accelerator
Holoscan	Pipeline / data flow	Medical AI pipelines suffer from jitter and compute–graphics interference	Provides deterministic end-to-end execution through partitioning, isolation, and admission control
SneakPeek	Edge / scheduler	Fixed edge resources cannot run the largest model for every request	Provides data-aware model selection and scheduling to allocate compute by urgency and utility

The central synthesis is that real-time healthcare AI should not be designed as a single-model service. It should be designed as a vertical system where clinical

urgency is detected at the edge, protected through the pipeline, and enforced at the GPU runtime.

Chapter 5

Performance Evaluation and Comparative Analysis

5.1 Evaluation Methodology

Because this thesis is based on three representative papers rather than a new implementation, the performance evaluation is a comparative analysis of the reported results. The goal is to understand how each paper supports the thesis topic and what metrics are most relevant for real-time healthcare AI.

The following metrics are central:

- **Preemption latency:** the delay before an urgent task can obtain GPU execution.
- **End-to-end latency:** the total time from input arrival to output delivery.
- **Maximum latency:** the worst observed latency, important for safety.
- **Tail spread and jitter:** the stability of latency distribution.
- **Throughput and utilization:** whether deadline protection wastes hardware.
- **Deadline violation rate:** how often requests miss timing requirements.
- **Utility:** a combined measure of accuracy, deadline satisfaction, and payoff.

5.2 REEF Performance

REEF reports that reset-based preemption reduces GPU preemption latency to less than 40 microseconds and achieves around a $10\times$ reduction compared with wait-based approaches. It also reports less than 2% end-to-end latency overhead for real-time tasks. Under concurrent real-time and best-effort inference, REEF can improve throughput by up to $7.7\times$ compared with conservative real-time-only reservation.

These results are important for the thesis because they show that preemption can improve both safety and utilization. Without preemption, a hospital system may reserve a GPU only for urgent tasks, which is safe but inefficient. With unsafe sharing, utilization improves but urgent tasks may be delayed. REEF shows a middle path: the GPU can be shared while still allowing urgent tasks to cut in quickly.

5.3 Holoscan Performance

The Holoscan paper evaluates medical AI workloads such as endoscopy tool tracking and ultrasound segmentation. It reports that the optimized design reduces maximum latency by approximately 21–30% compared with a single-GPU baseline. It also improves latency distribution flatness by approximately 17–25%, indicating tighter latency tails. Compared with a default multi-GPU design, the optimized system reports around 35% lower maximum latency and 42% better GPU utilization.

These results support the thesis argument that deterministic latency requires pipeline-level design. Simply adding more hardware does not guarantee predictable performance. The system must carefully assign compute and graphics roles, partition resources, and avoid oversubscription.

5.4 SneakPeek Performance

SneakPeek evaluates edge applications including fall detection, voice commands, and heart monitoring on a fixed GPU platform. Compared with earliest-deadline-first and simple model-selection baselines, SneakPeek reports about $2\times$ higher average schedule utility. It also reports near-zero deadline violations when grouped scheduling and short-circuit inference are enabled. The paper further shows that data awareness can reduce accuracy-estimation error under distribution shift.

For the thesis topic, these results show that edge scheduling must consider both deadlines and data. A scheduler that treats all requests equally may waste compute on easy samples and fail to allocate enough resources to clinically important samples. SneakPeek improves resource allocation by connecting input content to scheduling policy.

5.5 Cross-Paper Comparison

Table 5.1 summarizes the reported performance contributions.

TABLE 5.1: Performance contribution of the three systems.

System	Main metrics	Reported improvement	Meaning for health-care AI
REEF	Preemption latency, throughput, RT overhead	$<40\ \mu\text{s}$ preemption; up to $7.7\times$ throughput; $<2\%$ RT overhead	Urgent inference can interrupt background GPU work
Holoscan	Max latency, flatness, utilization	21–30% lower max latency; 17–25% better flatness; 42% better utilization vs default multi-GPU	Medical pipelines become more predictable under concurrency
SneakPeek	Utility, deadline violations, accuracy estimation	About $2\times$ utility; near-zero deadline violations with grouped scheduling and short-circuiting	Edge resources are focused on high-urgency or high-payoff data

The comparison shows that the three systems should not be ranked as alternatives. They solve different parts of the same problem. REEF protects urgent GPU execution. Holoscan protects end-to-end pipeline timing. SneakPeek protects resource allocation at the edge. A complete real-time healthcare AI system can benefit from all three.

5.6 Discussion of Practical Deployment

In a practical hospital deployment, these techniques would need to be combined carefully. Consider a surgical AI system. A video frame enters the system through a camera. The pipeline manager must guarantee that the frame reaches the inference engine and display path within a stable time window. If a critical frame arrives while the GPU is running a background task, preemption is needed. If the hospital edge server is overloaded, a scheduler must decide which model to run and whether to degrade gracefully.

This deployment scenario reveals why the three papers are correlated. SneakPeek-like data awareness can mark a request as urgent. Holoscan-like pipeline design can provide predictable data movement. REEF-like preemption can enforce the urgent request at the GPU level. Without any one of these layers, the system has a weakness. Without SneakPeek, the system may not know which request deserves priority. Without Holoscan, the urgent request may still be delayed by pipeline jitter. Without REEF, the urgent request may still wait behind a long GPU kernel.

5.7 Limitations of This Thesis Evaluation

This thesis does not implement a new integrated system. Therefore, it cannot claim that the numerical improvements from the three papers will directly multiply or transfer to all healthcare settings. The three papers use different workloads, hardware, and evaluation methods. Clinical deployment also requires validation with real data, clinician feedback, regulatory constraints, and fail-safe operation.

Nevertheless, the comparative evaluation is still useful because it identifies complementary system mechanisms and maps them to the end-to-end healthcare AI problem. The next step is to build an integrated prototype and evaluate it using realistic mixed clinical workloads.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This thesis studied end-to-end AI inference systems for real-time healthcare. The main conclusion is that real-time healthcare AI should be treated as a full-stack systems problem. When AI is used for live clinical intervention, accuracy is necessary but not sufficient. The system must also provide predictable timing, low tail latency, and resource-aware scheduling.

The thesis discussed three papers sequentially. REEF contributes kernel-level GPU preemption and shows how urgent inference can avoid being blocked by best-effort kernels. The deterministic Holoscan design contributes pipeline-level control and shows how medical streaming applications can reduce maximum latency and jitter through partitioning, isolation, and admission control. SneakPeek contributes edge-level scheduling and shows how data-aware model selection can improve utility and reduce deadline violations under fixed resources.

The correlation among the three papers is central to the thesis. SneakPeek decides which requests and models deserve resources. Holoscan ensures that admitted applications move through the medical pipeline predictably. REEF ensures that urgent tasks can obtain GPU execution quickly. Together, they form a vertical stack for preemptive, deterministic, and resource-aware healthcare AI inference.

6.2 Future Work

Several future directions remain.

6.2.1 Integrated System Prototype

The most important next step is to build an integrated prototype. Such a prototype could combine a data-aware scheduler, a deterministic Holoscan-like pipeline, and a preemptive GPU runtime. This would allow direct evaluation of whether the three mechanisms compose well.

6.2.2 Clinical Workload Traces

Future work should collect realistic traces from clinical environments. Surgical video, ICU monitoring, ultrasound streams, fall detection, and voice interaction all have different latency and accuracy requirements. Real traces would make scheduling and evaluation more meaningful.

6.2.3 Safety-Aware Degradation

When the system is overloaded, it should degrade safely. Possible strategies include switching to lighter models, reducing frame rate, prioritizing critical streams, or notifying clinicians that the AI output is delayed. These policies should be designed with clinical experts.

6.2.4 Hardware Portability

Some techniques discussed in this thesis depend on NVIDIA-specific features such as CUDA MPS or GPU runtime behavior. Future work should study whether the same principles can be applied to other accelerators and heterogeneous edge devices.

6.2.5 Clinical Validation

Finally, system metrics must be connected to clinical outcomes. Future studies should evaluate not only latency and throughput but also clinician trust, workflow impact, alarm usefulness, and safety thresholds.

6.3 Final Remark

Real-time healthcare AI is not only about building better models. It is about delivering the right model output to the right clinical user at the right time. This requires a system stack that can recognize urgency, protect deterministic execution, and enforce priority at the hardware level. The three papers discussed in this thesis provide complementary foundations for such a stack.

References

- Choi, J. et al. (2016). “Baymax: QoS Awareness and Increased Utilization for Non-Preemptive Accelerators in Warehouse Scale Computers”. In: *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- Crankshaw, Daniel et al. (2017). “Clipper: A Low-Latency Online Prediction Serving System”. In: *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 613–627.
- Dean, Jeffrey and Luiz André Barroso (2013). “The Tail at Scale”. In: *Communications of the ACM* 56.2, pp. 74–80.
- Frink, J., D. Frink, and A. Chandra (2025). “SneakPeek: Data-Aware Model Selection and Scheduling for Inference Serving on the Edge”. In: *Proceedings of the 2025 ACM Symposium on Cloud Computing*, pp. 748–761.
- Gujarati, Arpan et al. (2020). “Serving DNNs Like Clockwork: Performance Predictability from the Bottom Up”. In: *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 443–462.
- Han, Mingzhe et al. (2022). “Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences”. In: *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 539–558.
- Lo, David et al. (2015). “Heracles: Improving Resource Efficiency at Scale”. In: *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, pp. 450–462.
- Park, J. J. K., Youngjin Park, and Scott Mahlke (2015). “Chimera: Collaborative Preemption for Multitasking on a Shared GPU”. In: *Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 593–606.
- Romero, Francisco et al. (2021). “INFaaS: Automated Model-Less Inference Serving”. In: *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*, pp. 397–411.
- Shen, Haichen et al. (2019). “Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis”. In: *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 322–337.
- Sinha, S., S. Dwivedi, and M. Azizian (2024). “Towards Deterministic End-to-End Latency for Medical AI Systems in NVIDIA Holoscan”. In: *Proceedings of the 2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems (ICCPS)*, pp. 235–246.
- Tanasic, Ivan et al. (2014). “Enabling Preemptive Multiprogramming on GPUs”. In: *Proceedings of the 41st Annual International Symposium on Computer Architecture (ISCA)*, pp. 193–204.